HESTIM
ENGINEERING & BUSINESS SCHOOL

4e année Ingénierie Informatique et Intelligence Artificielle

Rapport d'avancement « LIVRABLE 2»

Sujet :

Système intelligent de détection précoce d'anomalies environnementales pour la sécurité des zones logistiques portuaires

---

Réalisé par :

- **Mohamed Traoré**
- **Mobio Aké Ivan Junior**
- **Junior Stevy Gatsoundou**
- **Kabré Yann Kévin**

Encadré par :

- **M.GILCHRIS**
- **Mme BOUHADDOU**

Année universitaire
2024/2025

Table des matières

Introduction 3
1 Diagrammes UML 3
1.1 Diagramme de cas d'utilisation (Use Case) 3
1.1.1 Acteurs identifiés 3
1.1.2 Principaux cas d'utilisation 3
1.2 Diagramme de classes 4
1.2.1 Classes principales 4
1.2.2 Relations entre classes 5
1.3 Diagramme de séquence 6
Scénario : Détection et traitement d'une anomalie de qualité de l'air 6
1.3.1 Scénario : Optimisation de l'éclairage public ..... **Erreur ! Signet non défini.**
1.4 Diagramme d'activité 7
1.5 Diagramme d'état-transition ..... **Erreur ! Signet non défini.**
2 Architecture Hardware 9
2.1 Vue d'ensemble de l'architecture matérielle 9
2.1.1 Couche Terrain – Noeuds IoT 9
2.1.2 Couche Edge – Passerelle locale 9
2.1.3 Couche Traitement Local – Infrastructure on-premise 9
2.2 Schéma de connectivité et protocoles de communication 10
2.3 Spécifications techniques des composants 10
3 Architecture Software 12
3.1 Architecture générale du logiciel 12
3.1.1 Couche Acquisition (Firmware embarqué) 12
3.1.2 Couche Communication (Middleware IoT) 12
3.1.3 Couche Traitement (Backend IA) 12
3.1.4 Couche Stockage (Data Layer) 13
3.1.5 Couche Présentation (Frontend) 13
3.2 Architecture du pipeline de traitement des données 13
3.3 Architecture du modèle Machine Learning (traitement PC local) 13
3.3.1 Processus d'entraînement et de déploiement 13
3.3.2 Caractéristiques du modèle ML (PC local) 14
3.4 Sécurité et fiabilité de l'architecture logicielle 14
4 Synthèse et validation de l'architecture 15
4.1 Cohérence architecturale 15
4.2 Perspectives pour le Livrable Final 15

Conclusion..... 15

Introduction

Dans la continuité du Livrable 1, qui a posé les bases conceptuelles et technologiques du projet Système IoT intelligent pour la gestion urbaine dans les Smart Cities, ce deuxième livrable constitue l'étape de formalisation architecturale et de modélisation du système.

Il présente les diagrammes UML qui modélisent le comportement et la structure du système, ainsi que les schémas d'architectures hardware et software qui décrivent les composants physiques et logiciels ainsi que leurs interactions.

Ces livrables représentent une étape essentielle dans le cycle de développement du projet, car ils permettent de valider la cohérence de la solution proposée avant son implémentation effective.

1 Diagrammes UML

La modélisation UML (Unified Modeling Language) du système IoT intelligent pour la gestion urbaine couvre l'ensemble des aspects statiques et dynamiques de la solution. Les diagrammes présentés ci-après ont été élaborés conformément aux normes UML 2.5 et reflètent l'architecture fonctionnelle du système.

1.1 Diagramme de cas d'utilisation (Use Case)

Le diagramme de cas d'utilisation décrit les interactions entre les différents acteurs du système et les fonctionnalités offertes par la plateforme IoT Smart City.

1.1.1 Acteurs identifiés

- Citoyen : utilisateur final qui consulte les informations environnementales via une application mobile ou un portail web.
- Opérateur urbain : gestionnaire chargé de superviser les alertes, analyser les tableaux de bord et déclencher des actions correctives.
- Système IoT (capteurs) : acteur externe représentant les noeuds de capteurs qui collectent et transmettent les données.
- Module IA / PC local : composant qui traite les données reçues des capteurs IoT sur un PC local et génère des décisions automatiques.
- Administrateur système : responsable de la configuration, la maintenance et la sécurité de l'infrastructure.

1.1.2 Principaux cas d'utilisation

- Collecter les données environnementales (qualité de l'air, trafic, éclairage, déchets)
- Traiter les données en temps réel via le module ML sur PC local
- Détecter les anomalies et générer des alertes automatiques
- Visualiser les indicateurs sur un tableau de bord interactif

- Configurer les seuils d’alerte et les règles de décision
- Gérer les données historiques et générer des rapports
- Administrer les nœuds IoT (configuration, mise à jour, diagnostic)

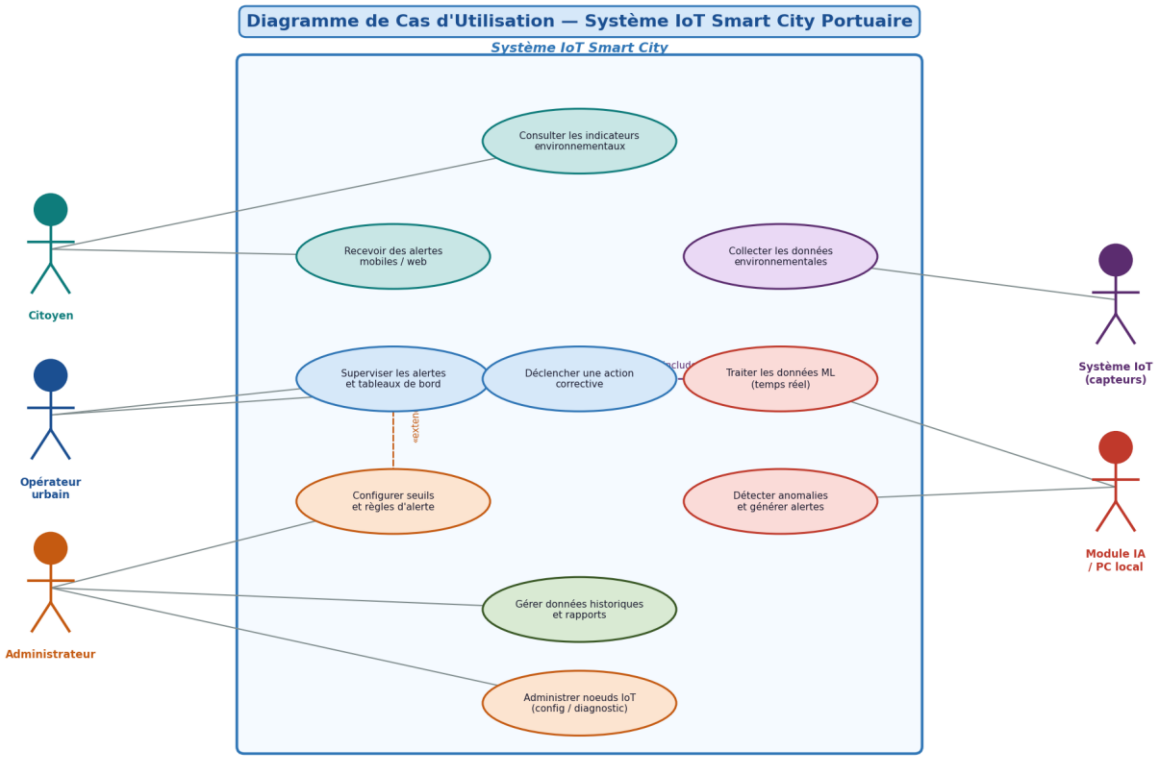


Figure 11 — Diagramme de Cas d'Utilisation (Use Case)

1.2 Diagramme de classes

Le diagramme de classes modélise la structure statique du système en définissant les entités principales, leurs attributs, leurs méthodes et les relations qui les unissent.

1.2.1 Classes principales

Capteur (Sensor)

- Attributs : id, type, localisation, état, fréquenceéchantillonnage
- Méthodes : lireDonnee(), calibrer(), diagnostiquer(), envoyerDonnee()

DonnéeMesure (SensorData)

- Attributs : valeur, unité, timestamp, capteurId, qualitéSignal
- Méthodes : valider(), normaliser(), serialiser()

ModuleIA (AIModule)

- Attributs : modèleML, seuils, historiqueDonnées, étatPrédiction
- Méthodes : analyserFlux(), détecterAnomalies(), prédire(), mettreAJourModèle()

Alerte (Alert)

- Attributs : id, type, niveau (critique/majeur/mineur), message, timestamp, coordonnées
- Méthodes : créer(), notifier(), escalader(), acquitter()

TableauDeBord (Dashboard)

- Attributs : widgetsList, étatActualisation, filtresActifs, utilisateurCourant
- Méthodes :afficherIndicateurs(), exporterRapport(), filtrerZone(), actualiserTempsRéel()

Utilisateur (User)

- Attributs : id, nom, rôle (citoyen / opérateur / admin), permissions, préférences
- Méthodes : s'authentifier(), consulterAlertes(), configurerParamètres()

1.2.2 Relations entre classes

- Un Capteur génère plusieurs DonnéeMesure (composition 1..n)
- Le ModuleIA analyse plusieurs DonnéeMesure et produit des Alertes (dépendance)
- Le TableauDeBord agrège les Alertes et les DonnéeMesure (agrégation)
- Un Utilisateur accède au TableauDeBord selon son rôle (association avec stéréotype)

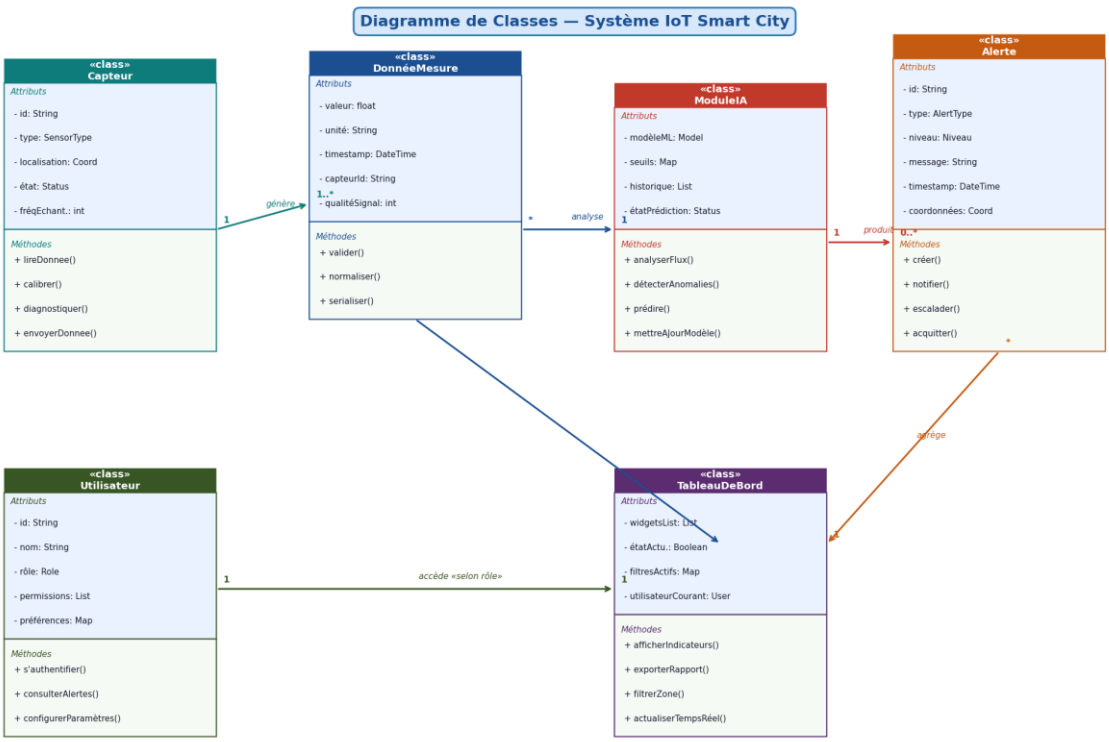


Figure 22 — Diagramme de Classes UML

1.3 Diagramme de séquence

Le diagramme de séquence illustre les interactions entre les composants du système pour un scénario de traitement d'anomalie de bout en bout.

Scénario : Détection et traitement d'une anomalie de qualité de l'air

Les étapes du scénario sont les suivantes :

- 1. Le capteur MQ-135 mesure la concentration de CO2 et de particules fines en continu.
- 2. Les données brutes sont prétraitées localement par le microcontrôleur ESP32 (filtrage, normalisation).
- 3. Le module ML sur PC local analyse la série temporelle reçue et détecte un écart anormal.
- 4. Un message d'alerte est publié sur le broker MQTT via le protocole de communication IoT.
- 5. Le backend serveur reçoit la notification, enrichit l'alerte avec les métadonnées de localisation.
- 6. Le système de notification envoie une alerte push à l'opérateur urbain concerné.
- 7. Le tableau de bord se met à jour en temps réel pour afficher la zone affectée.
- 8. L'opérateur consulte le détail, acquitte l'alerte et déclenche une action corrective.

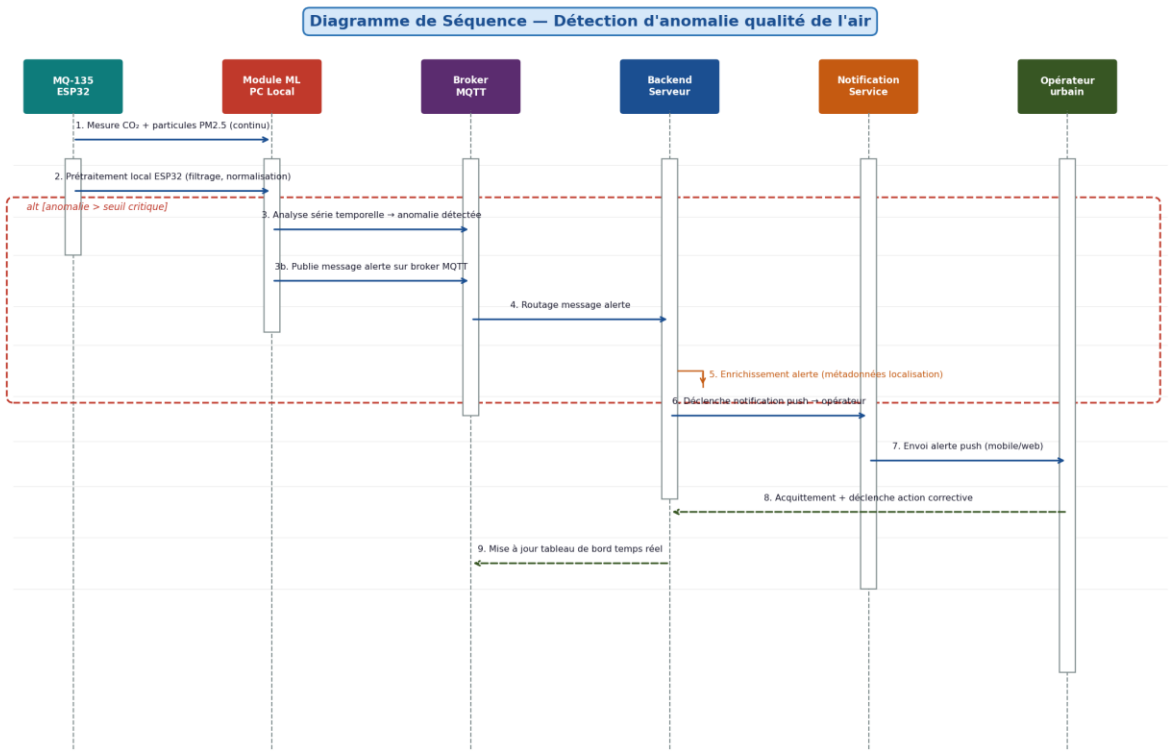


Figure 3Diagramme de Séquence — Détection anomalie qualité de l'air

1.4 Diagramme d’activité

Le diagramme d’activité représente le flux de traitement des données IoT, de la collecte jusqu’à la prise de décision, en mettant en évidence les branchements conditionnels et les traitements parallèles.

Les étapes clés du flux d’activité sont :

- Début : initialisation des capteurs et établissement des connexions réseau
- Acquisition : lecture périodique des valeurs capteurs
- [Si anomalie détectée] : activation du pipeline d’alerte
- [Sinon] : stockage des données et mise à jour du tableau de bord
- Traitement IA parallèle : analyse prédictive et optimisation des ressources urbaines
- Actions automatisées : commande des actionneurs (éclairage, feux, gestion déchets)
- Boucle itérative : retour vers l’étape d’acquisition

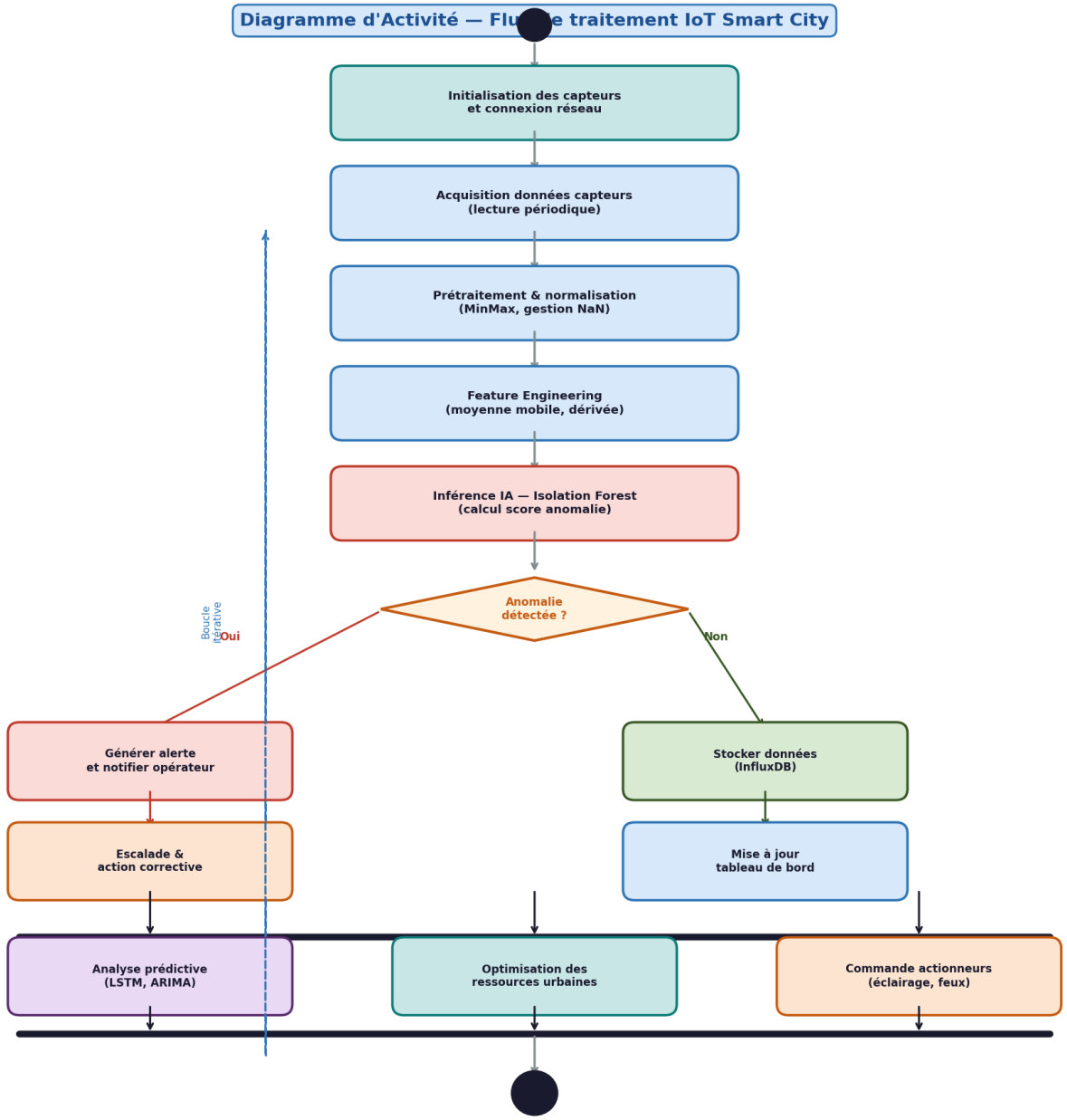


Figure 44 — Diagramme d'Activité — Flux de traitement IoT

2 Architecture Hardware

L'architecture matérielle du système IoT Smart City repose sur une organisation hiérarchique en trois couches : la couche terrain (capteurs et actionneurs), la couche edge (traitement local) et la couche traitement local (traitement centralisé et visualisation on-premise).

2.1 Vue d'ensemble de l'architecture matérielle

L'architecture matérielle est organisée selon trois niveaux hiérarchiques complémentaires, garantissant une collecte robuste, un traitement rapide et une supervision centralisée.

2.1.1 Couche Terrain – Noeuds IoT

Chaque noeud IoT est constitué des éléments suivants :

- Microcontrôleur ESP32 : unité centrale de traitement embarqué, WiFi/Bluetooth intégré, collecte et transmission des données capteurs
- Capteurs environnementaux : MQ-135 (qualité de l'air / CO2), DHT22 (température et humidité), GP2Y1010AU0F (particules fines)
- Capteurs de mobilité : HC-SR04 (ultrasons pour détection flux), capteur LDR (luminosité ambiante)
- Actionneurs : module relais pour commande d'éclairage LED, buzzer d'alerte
- Alimentation : panneau solaire + batterie LiPo pour autonomie énergétique
- Communication : module LoRa SX1276 pour transmission longue portée (zones sans WiFi)

2.1.2 Couche Edge – Passerelle locale

La passerelle edge assure l'agrégation et le prétraitement des données avant transmission vers le serveur de traitement local :

- Raspberry Pi 4 Model B : processeur quad-core 1.8 GHz, 4 Go RAM, OS Linux embarqué
- Broker MQTT Mosquitto : gestion des messages IoT en mode publish/subscribe
- Module de prétraitement : nettoyage et agrégation des données avant transmission au PC local
- Stockage local : carte microSD 64 Go pour buffer de données en cas de coupure réseau
- Interface réseau : WiFi, Ethernet, module LoRa concentrateur

2.1.3 Couche Traitement Local – Infrastructure on-premise

- Serveur de traitement local : machine dédiée on-premise (16 Go RAM, GPU pour entraînement et inférence ML)
- Base de données temporelle : InfluxDB pour le stockage des séries temporelles capteurs
- Serveur d'application : Node.js / Python FastAPI pour les API REST
- Interface de visualisation : Grafana pour les tableaux de bord temps réel

- Architecture entièrement on-premise : traitement des données et inférence ML réalisés localement, sans dépendance à un service cloud externe

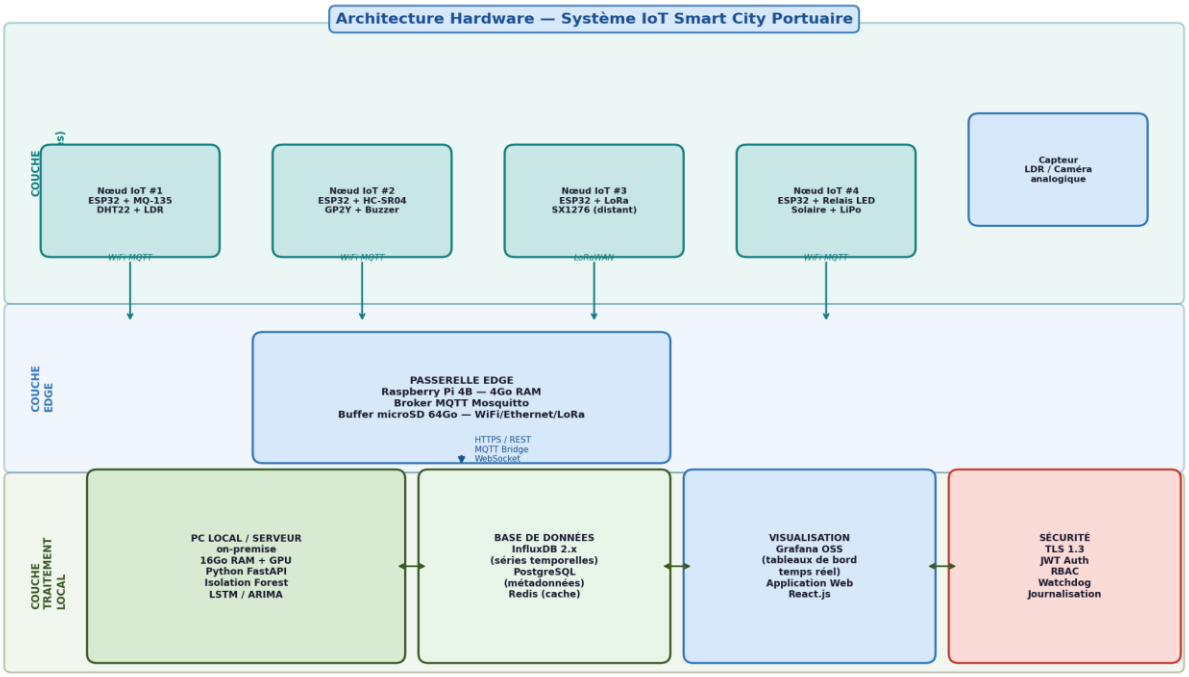


Figure 5---Architecture Hardware — Trois couches (Terrain / Edge / Traitement local)

2.2 Schéma de connectivité et protocoles de communication

Les protocoles de communication retenus pour chaque niveau de l'architecture sont :

Niveau capteur → passerelle

- MQTT over WiFi (ESP32 → Raspberry Pi) : protocole léger, faible consommation, QoS configurable
- LoRaWAN (noeuds distants → concentrateur) : portée jusqu'à 10 km, débit faible mais suffisant
- I2C / SPI (capteurs → ESP32) : bus de communication interne entre capteurs et micro-contrôleur

Niveau passerelle → serveur local

- HTTPS / REST API : transmission sécurisée des données agrégées vers le serveur local de traitement
- MQTT Bridge : relais des messages MQTT locaux vers le broker on-premise
- WebSocket : communication temps réel pour les alertes urgentes

2.3 Spécifications techniques des composants

Le tableau ci-dessous récapitule les principaux composants matériels retenus et leurs caractéristiques techniques :

Tableau 1. Composants matériels et spécifications

Composant	Modèle	Caractéristiques principales
Microcontrôleur	ESP32 WROOM-32	Dual-core 240 MHz, WiFi+BT, 4 Mo Flash, collecte et envoi des données capteurs
Passerelle Edge	Raspberry Pi 4B	Quad-core 1.8 GHz, 4 Go RAM, WiFi, Ethernet, USB 3.0
Capteur air	MQ-135	Détection CO2, NH3, benzène, fumée, plage 10-300 ppm
Capteur T°/Hum.	DHT22	Temp. -40 à +80°C ±0.5°C, Humidité 0-100% ±2-5%
Capteur particules	GP2Y1010AU0F	Détection PM2.5, plage 0-0.5 mg/m³, sortie analogique
Module LoRa	SX1276	Fréquence 868/915 MHz, portée 10 km, débit 0.3-37.5 kbps
Base de données	InfluxDB 2.x	Séries temporelles, compression haute performance, API Flux
Visualisation	Grafana OSS	Tableaux de bord temps réel, alerting intégré, plugins IoT

3 Architecture Software

L'architecture logicielle du système est conçue selon le patron architectural en couches (Layered Architecture), garantissant la séparation des responsabilités, la maintenabilité du code et l'évolutivité de la solution.

3.1 Architecture générale du logiciel

L'architecture logicielle complète est structurée en cinq couches fonctionnelles distinctes :

3.1.1 Couche Acquisition (Firmware embarqué)

Développée en langage C/C++ sur la plateforme Arduino/ESP-IDF pour les noeuds ESP32 :

- Drivers capteurs : librairies de bas niveau pour chaque type de capteur (DHT, MQ, GP2Y)
- Gestionnaire d'échantillonnage : scheduler périodique configurable par capteur
- Prétraitement embarqué : filtrage médian, détection de valeurs aberrantes
- Client MQTT : connexion sécurisée (TLS), collecte des mesures capteurs et publication sur les topics dédiés vers le PC local

3.1.2 Couche Communication (Middleware IoT)

Cette couche gère les échanges de messages entre les différents composants du système :

- Broker MQTT Eclipse Mosquitto : routage des messages, gestion des souscriptions, persistence
- Architecture de topics : /smartcity/{zone}/{type_capteur}/{id_capteur}/data
- Module de validation : vérification du schéma JSON des messages entrants
- Gestionnaire de connexions : détection de déconnexion, gestion des Last Will Testament

3.1.3 Couche Traitement (Backend IA)

Développée en Python avec le framework FastAPI pour l'exposition des services :

- Service d'ingestion : réception, validation et stockage des données dans InfluxDB
- Service de détection d'anomalies : modèle Isolation Forest (scikit-learn), score en temps réel
- Service de prédiction : séries temporelles (ARIMA / LSTM léger) pour anticipation
- Moteur de règles : évaluation des conditions d'alerte, escalade automatique
- API REST : endpoints pour consultation des données, gestion des alertes et configuration

3.1.4 Couche Stockage (Data Layer)

- InfluxDB 2.x : stockage optimisé des séries temporelles avec politique de rétention configurable
- PostgreSQL : stockage relationnel des métadonnées (capteurs, zones, utilisateurs, alertes)
- Redis : cache en mémoire pour accès rapide aux dernières valeurs et gestion des sessions
- Système de fichiers : archivage des modèles ML entraînés et des rapports générés

3.1.5 Couche Présentation (Frontend)

- Tableau de bord Grafana : visualisation temps réel, cartes interactives, graphiques de séries
- Application web (React.js) : portail citoyen et interface opérateur avec authentification JWT
- API WebSocket : mise à jour push des alertes et indicateurs sans rafraîchissement manuel

3.2 Architecture du pipeline de traitement des données

Le pipeline de traitement suit un flux unidirectionnel structuré en étapes successives :

- Étape 1 – Collecte : acquisition périodique des données capteurs (fréquence paramétrable de 1 sec à 5 min)
- Étape 2 – Prétraitement : nettoyage, normalisation MinMax, gestion des valeurs manquantes
- Étape 3 – Feature Engineering : extraction de caractéristiques (moyenne mobile, écart-type local, dérivée)
- Étape 4 – Inférence IA : passage dans le modèle de détection d'anomalies, calcul du score
- Étape 5 – Décision : comparaison au seuil dynamique, classification (normal / suspect / anomalie)
- Étape 6 – Stockage : persistance dans InfluxDB avec tags de zone et de niveau d'alerte
- Étape 7 – Notification : déclenchement des alertes et mise à jour du tableau de bord

3.3 Architecture du modèle Machine Learning (traitement PC local)

Le modèle Machine Learning est exécuté sur un PC local. Les données collectées par les noeuds ESP32 sont transmises via MQTT au PC, qui se charge de l'inférence, de la détection d'anomalies et de la prise de décision. Cette approche permet d'exploiter pleinement les ressources matérielles du PC (CPU/GPU, RAM) sans aucune contrainte d'embarquement.

3.3.1 Processus d'entraînement et de déploiement

- Entraînement en local : Python + TensorFlow / Keras avec données simulées et réelles sur machine dédiée on-premise
- Optimisation : hyperparameter tuning, validation croisée, sélection des features pertinentes
- Sauvegarde : export du modèle entraîné au format pickle (.pkl) ou joblib pour réutilisation

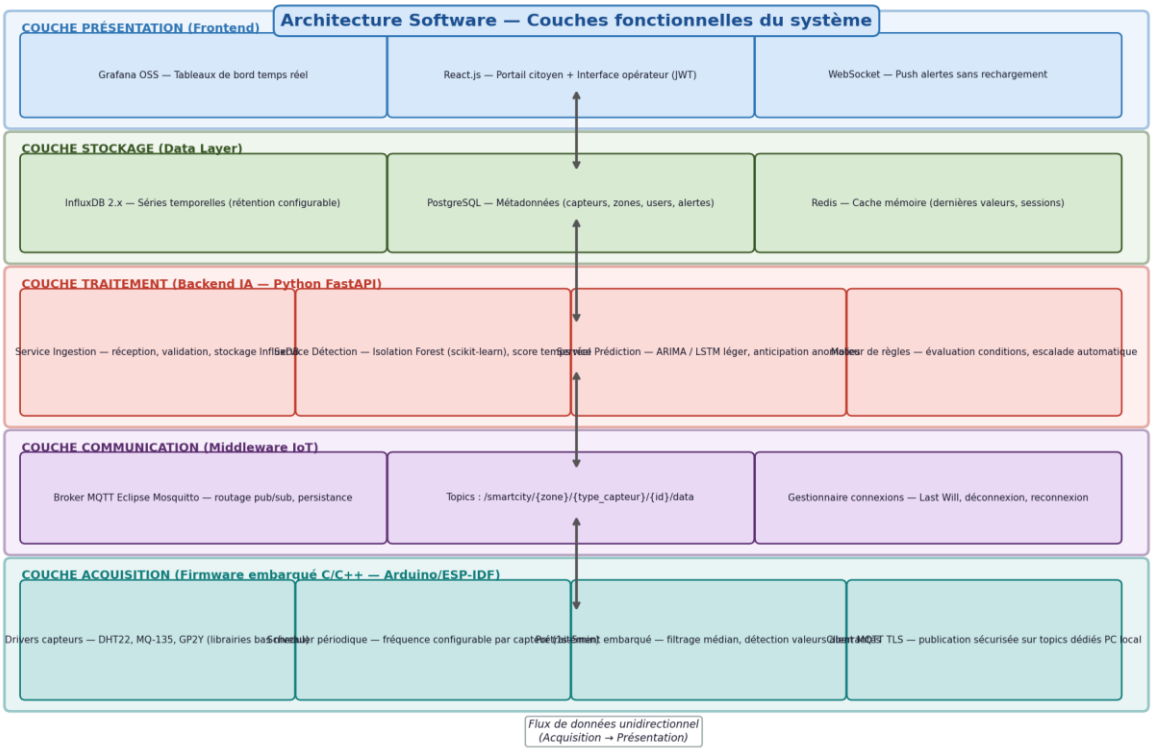
- Déploiement : chargement du modèle sur le PC local via un service Python (FastAPI) qui reçoit les données MQTT et exécute l'inférence en temps réel
- Mise à jour du modèle : ré-entraînement périodique en local avec les nouvelles données accumulées pour améliorer les performances

3.3.2 Caractéristiques du modèle ML (PC local)

- Architecture : Isolation Forest (scikit-learn) pour la détection non supervisée d'anomalies, complété par un LSTM pour l'analyse de séries temporelles
- Ressources PC : exploitation complète du CPU/GPU disponible, sans contrainte de taille de modèle ni de mémoire
- Latence d'inférence : inférieure à 50 ms par cycle d'analyse sur PC local
- Consommation mémoire : gestion dynamique par le système d'exploitation du PC, sans limitation matérielle critique

3.4 Sécurité et fiabilité de l'architecture logicielle

- Chiffrement TLS 1.3 pour toutes les communications MQTT et HTTP entre composants
- Authentification JWT (JSON Web Token) pour l'accès aux API et au tableau de bord
- Gestion des droits RBAC (Role-Based Access Control) : citoyen / opérateur / administrateur
- Watchdog timer sur les noeuds ESP32 : redémarrage automatique en cas de blocage
- Buffer local sur passerelle : tolérance aux pannes réseau avec ré-émission différée
- Journalisation système : traçabilité complète des événements et des décisions IA



4 Synthèse et validation de l'architecture

Les diagrammes UML et les schémas d'architectures présentés dans ce livrable constituent le référentiel de conception du système IoT intelligent pour la gestion urbaine.

4.1 Cohérence architecturale

La cohérence entre les différentes vues UML et les schémas d'architecture a été vérifiée :

- Les classes UML correspondent directement aux composants logiciels décrits dans l'architecture software
- Les acteurs du diagramme de cas d'utilisation sont cohérents avec les rôles définis dans le système de sécurité
- Les états du diagramme d'état-transition sont alignés avec le cycle de vie décrit dans le firmware embarqué
- Le diagramme de séquence valide la faisabilité du pipeline de traitement des données

4.2 Perspectives pour le Livrable Final

Sur la base de cette architecture formalisée, les prochaines étapes du projet porteront sur :

- L'implémentation effective du firmware ESP32 et la configuration du broker MQTT
- Le développement du backend Python (FastAPI + ML + InfluxDB)
- L'entraînement et la validation du modèle ML (Isolation Forest + LSTM) sur les données simulées en local
- La mise en place du tableau de bord Grafana avec les dashboards Smart City
- Les tests d'intégration et la validation des scénarios d'anomalie définis dans les diagrammes de séquence

Conclusion

Ce Livrable 2 a permis de formaliser l'ensemble de l'architecture du système IoT intelligent pour la gestion urbaine des Smart Cities, à travers une double approche : la modélisation comportementale et structurelle via les diagrammes UML, et la description précise des composants matériels et logiciels via les schémas d'architecture.

Les diagrammes UML — cas d'utilisation, classes, séquences, activités et états-transitions — offrent une vision complète et cohérente du comportement du système. L'architecture matérielle en trois couches (terrain, edge, traitement local) et l'architecture logicielle en cinq niveaux (acquisition, communication, traitement, stockage, présentation) définissent un cadre technique robuste, évolutif et industrialisable.

Ces livrables constituent la base de référence pour la phase d'implémentation qui fera l'objet du prototype fonctionnel présenté lors de la soutenance finale.